Object Oriented Programming

UPES
UNIVERSITY OF TOMORROW

Dr. Suryakant Prasad

Methods Overloading

Method Overloading is a feature in Java that allows a class to have more than one methods having same name, but with different signatures (Each method must have different number of parameters or parameters having different types and orders).

Advantage:

- Method Overloading increases the readability of the program.
- Provides the flexibility to use similar method with different parameters.

Three ways to overload a method

In order to overload a method, the argument lists of the methods must differ in either of these:

- Number of parameters. (Different number of parameters in argument list)

  For example: This is a valid case of overloading

  add(int, int)

  add(int, int, int)

- Data type of parameters. (Difference in data type of parameters)

  For example:

  add(int, int)

  add(int, float)

- Sequence of Data type of parameters.
  For example:
  add(int, float)
  add(float, int)

Rules for Method Overloading:

- First and important rule to overload a method in java is to change method signature.

- Return type of method is never part of method signature, so only changing the return type of method does not amount to method overloading.

To find the Minimum of given numbers:

```
public class OverloadingCalculation1
{
public static void main(String[] args)
{
int a = 11;
int b = 6;
int c = 3;
double x = 7.3;
double y = 9.4;
int result1 = minFunction(a, b, c);
double result2 = minFunction(x, y);
double result3 = minFunction(a, x);
System.out.println("Minimum("+a+","+b+","+c+") = " + result1);
System.out.println("Minimum("+x+","+y+") = " + result2);
System.out.println("Minimum("+a+","+x+") = " + result3);
}
```

```
public static int minFunction(int n1, int n2, int n3)
{
int min;
int temp = n1<n2? n1 : n2;
min = n3 < temp? n3 : temp;
return min;
}
public static double minFunction(double n1, double n2)
{
double min;
if (n1 > n2) min = n2;
else min = n1;
return min;
}

public static double minFunction(int n1, double n2)
{
double min;
if (n1 > n2) min = n2;
else min = n1;
return min;
}
}
```

```
Output
Minimum(11,6,3) = 3
Minimum(7.3,9.4) = 7.3
Minimum(11,7.3) = 7.3
```

Note:- Method overloading is not possible by changing the return type of the method because of ambiguity that may arise while calling the method with same parameter list with different return type.

```
class Add
{
static int sum(int a, int b)
{
        return a+b;
}
static float sum(int a, int b)
{
        return a+b;
}
public static void main(String arg[])
{
        System.out.println(sum(10,20));
        System.out.println(sum(15,25));
}
}
```

Raise the error

Static Members in Java

- Static members belong to the class, not to objects.
  - Static variables
  - Static methods
  - Static blocks

- Declared using the static keyword.

Static variables

- Declared using static keyword
- Belongs to class, not objects
- Only one copy shared by all objects
- Access using ClassName.variable
- Lifetime: till program ends
- Default values provided by JVM

```
class Student {
    static String college = "NIT";
    int rollNo;
}

System.out.println(Student.college);
```

Difference between instance variables and static variables (class variables)

Instance variables	Static variables
An instance variable is a variable whose separate copy is available to each object.	A static variable is a variable whose single copy in memory is shared by all objects.
Instance variables are created in the objects on heap memory.	Static variables are stored on method area.

- Since, instance variable will have a separate copy in each object, when the value of an instance variable is modified in an object, it does not affect the instance variables in other objects.
- Since a static variable (class variable) will have only one copy in memory and that is shared by all the objects, any modification to it will also affect other objects.
- Observe the output of the following programs.

```
class methodsDemo1 {
   private int s=10, t=20;
   static int a=5;
 void update()
    {
     s++; --t; ++a;
    }
 void read()
 {
  System.out.println("s= "+s+" "+"t= "+t+" "+"a= "+a);
 }
}
public class HellowWorld {
public static void main(String[] args){
    methodsDemo1 o1=new methodsDemo1();
    methodsDemo1 o2=new methodsDemo1();
    o1.update();
    o1.read();
    o2.read();
   }}
```

```
class methodsDemo1 {
    int s=10, t=20; // no private here
    static int a=5;
void read()
 {
  System.out.println("s= "+s+" "+"t= "+t+" "+"a= "+a);
 }
}
public class HellowWorld {
public static void main(String[] args){
    methodsDemo1 o1=new methodsDemo1();
    methodsDemo1 o2=new methodsDemo1();
    o1.s++;--o1.t;o1.a++;// methodsDemo1.a++;
    o1.read();
    o2.read();
   }
 }
```

Update instance variables from main method

Static Methods

Method is declared as static

```
class methodsDemo1 {

 static double sum(double x, double y)
    {
     double z=x+y;
     return z;
    }
}
public class HellowWorld {
public static void main(String[] args){

System.out.println("sum  is "+methodsDemo1.sum(3.5,5));
   }
 }
```

Static methods cannot access instance variables,
but they can access static variables.

```
class methodsDemo1 {
  int a=5;
 static double sum(double x, double y)
    {
     double z=x+y+a;
     return z;
    }
}
public class HellowWorld {
public static void main(String[] args){

System.out.println("sum  is "+methodsDemo1.sum(3.5,5));
   }
 }
```

Raises an error

```
class methodsDemo1 {
  static int a=5;
 static double sum(double x, double y)
    {
     double z=x+y+a;
     return z;
    }
}
public class HellowWorld {
public static void main(String[] args){

System.out.println("sum  is "+methodsDemo1.sum(3.5,5));
   }
 }
```

Static Methods

- A static method is a method that does not act upon instance variables of a class.
- A static method is declared by using the keyword static.
- Static methods are called using

  `Classname.methodname().`
- The reason why static methods can not act on instance variables is that the JVM first executes the static methods and then only it creates the objects.
- Since the objects are not available at the time of calling the static methods, the instance variables are also not available.
- The other name for static variable is class variable and for static method is class method.

Program to add two numbers

```
class methodsDemo1 {
double sum(double x, double y)
    {
     double z=x+y;
     return z;
    }
}
public class HellowWorld {
public static void main(String[] args){
    methodsDemo1 abc=new methodsDemo1();
    System.out.println("sum  is "+abc.sum(3.5,5));
   }
 }
```

Observation

- The method does not act on any instance variables.
- Hence, no need to create any object.
- Call the method with class name (for this we need to declare the method as static).

```
class methodsDemo1 {
 double sum(double x, double y)
    {
     double z=x+y;
     return z;
    }
}
public class HellowWorld {
public static void main(String[] args){
    System.out.println("sum  is "+methodsDemo1.sum(3.5,5));
   }
 }
```

Raises an error

The execution sequence of JVM

- static blocks
- static methods (`main()` is a static method)
- create objects needed by program
- instance methods

Static block

- A static block is a block of statements declared as static as follows

```
static{

    statements
}
```

- JVM executes first static block and then `main()` method (i.e., static method)

```
public class HellowWorld {
    static {
        System.out.println("I am always first!");
    }
public static void main(String[] args){
System.out.println("I am second if static block is present!");
   }
 }
```

Instance Methods

- To call instance method, object is needed because instance variables are contained in the object.
- Syntax to call instance method is

```
objectname.method_name();
```

- These methods can access directly both instance variables and static variables.

```
class methodsDemo1 {
   private int s;
  methodsDemo1()throws IOException
  {
      s=10;
      read();
  }
 void read()throws IOException
 {  int s=5;
 BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
System.out.println("enter an integer: ");
  int n=Integer.parseInt(br.readLine());
int b=this.s+n;
  System.out.println("sum is "+b);
 }
}
public class HellowWorld {
public static void main(String[] args)throws IOException{
    methodsDemo1 o1=new methodsDemo1();
}}
```

Instance method is calling from constructor, but not from main() method

Two types

1. Accessor methods
   - Simply access or read instance variables.
   - Do not modify the instance variables.
2. Mutator methods
   - Not only access the instance variables but also modify them.

Constructors

- A constructor is similar to a method that is used to initialize the instance variables.
- The sole purpose of a constructor is to initialize the instance variable.
- A constructor has the following characteristics.
  - constructor's name = class name
  - Syntax is

```
constructor_name()
{

}
```

- A constructor may have or may not have parameters
  - default constructor – does not have any parameters

```
constructor_name()
{

}
```

  - parameterized constructor – has one or more parameters

Example

```
constructor_name(String S, int n)
{

}
```

- A constructor does not return any value, not even void. Hence, we should not write void before the constructor name.
- A constructor is called concurrently when the object creation is going on.
  - JVM first allocates memory for the object and then executes the constructor to initialize the instance variables.
  - By the time, object creation is completed, the constructor execution is also completed.
- A constructor is called and executed only once per object.

```
class Student
{
    private String name;
    private int roll;
    Student()
    {
      name="ysr";
      roll=35;
    }
    void details()
    {
     System.out.println("my name is "+name);
     System.out.println("my roll no is "+roll);
    }
}
public class HellowWorld {

public static void main(String[] args){
Student abc=new Student();
abc.details();
Student xyz=new Student();
xyz.details();
}
}
```

Both the objects printing the same data

- We can observe the following from the output.
  - xyz object is also printing the data of the abc object.
- We want xyz object should get xyz student data.
- We can achieve this by using parameterized constructor.

```java
class Student
{
    private String name;
    private int roll;
    Student()
    {
      name="ysr";
      roll=35;
    }
    Student(String s, int n)
    {
      name=s;
      roll=n;
    }
    void details()
    {
     System.out.println("my name is "+name);
     System.out.println("my roll no is "+roll);
    }

}
```

```java
public class HellowWorld {

public static void main(String[] args){
Student abc=new Student();
abc.details();
Student xyz=new Student("rao", 100);
xyz.details();
}
}
```

- A constructor is automatically called and executed at the time of creating an object.
- While creating an object, if nothing is passed to the object, the default constructor is called and executed.
- If some values are passed to the object, then the parameterized constructor is called and executed.

- Writing two or more constructors with the same name but with difference in the parameters is called constructor overloading.
  - Such constructors are useful to perform different tasks.
- When the programmer does not write any constructor in a class, then Java compiler writes a default constructor and initializes the instance variables with the default values.

Default constructor	Parameterized constructor
Default constructor is useful to initialize all objects with same data.	parameterized constructor is useful to initialize each object with different data.
Default constructor does not have any parameters.	parameterized constructor will have 1 or more parameters.
When data is not passed at the time of creating an object, default constructor is called.	When data is passed at the time of creating an object, parameterized constructor is called.

Method and constructor Overloading - Java Lab Practice

Note: Analyze the program execution, evaluate the output generated, and apply your understanding to explain the observed behavior.

1. Write a Java program that overloads a method calculate() to perform:
   - Sum of two integers
   - Sum of three integers
   - Sum of two double values
   - Concatenation of two strings

   Display the result of each method call from main().

2. Create a class Employee with instance variables: name, id, salary.
   - Default constructor initializes with fixed values.
   - Parameterized constructor initializes with user-provided values.

   Create two objects and display employee details.

3. Create a class Counter with:
   - static variable count
   - instance variable id

   Increment both variables inside constructor.
   Create 3 objects and display values of count and id for each object.
   Explain the output.

4. Create a class that has one instance variable and one static variable.
   Create a static method trying to access both variables.
   Observe and fix the compilation error.
   Explain why static methods cannot access instance variables.

5. Write a program containing:
   - Static block
   - Instance block
   - Constructor
   - main() method

   Print statements in each block and observe execution order.

6. Write a program with:
   void display(int a, float b)
   void display(float a, int b)
   Call display(10, 10);
   Does it compile? Explain why ambiguity occurs.

7. Create a class with two instance variables and one static variable.
   Modify all variables from main() using object reference.
   Create two objects and observe the changes.
   Explain why static variable changes affect all objects.

8. Create a class where constructor calls an instance method.
   Instance method reads input using BufferedReader and adds value to instance variable.
   Create object from main() and observe execution.

9. Create a class with no constructor and one instance variable.
   Create object and print variable value.
   Then define your own constructor.
   Observe difference and explain default constructor role.

10. Create a class Student with name, roll number and static variable totalStudents.
    Overload constructors (default and parameterized).
    Each time object is created, increment totalStudents.
    Display total number of students created.